Foundations

Neurons, activations, MLPs, losses, and backpropagation

CSE752: Deep Neural Networks

Moin Moistakim
Department of CSE, School of Data and Sciences, BRAC University

Week 1 | Fall 2026

Today

Theory

- The neuron as an affine map plus a nonlinearity
- Why depth needs activations (XOR)
- Fully connected layers and parameter shapes
- MSE, binary cross-entropy, softmax cross-entropy
- Computation graphs and the chain rule
- Backpropagation in scalars, then matrices

Practice in this room

- One logistic neuron, computed by hand
- A 2–2–1 ReLU net, forward and backward
- A dead-ReLU unit and what its gradient does
- Four problems (solutions linked from each slide)

After class

- Goodfellow Ch. 6; Prince Ch. 3–4
- Home Assignment W1: NumPy MLP, autograd check
- Quiz 1 is this week

Learning objectives

By the end of this lecture you can:

1. Write the forward map of a multilayer perceptron, including every shape.
2. Differentiate sigmoid, tanh, and ReLU, and state where each gradient vanishes.
3. Derive $\partial L/\partial z = \hat{y} - y$ for binary cross-entropy with a sigmoid.
4. Backpropagate a scalar loss through one hidden layer using outer products.
5. Match the column-vector formulas to the row-batch NumPy layout in Home Assignment.

Course outcomes touched today: CO1 (explain the primitives), CO2 (the backprop you will code).

Notation used on the board

Symbol	Meaning
$x = a^{(0)} \in \mathbb{R}^d$	one input, a column
$W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$	weights of layer ℓ (out $\times$ in)
$b^{(\ell)} \in \mathbb{R}^{n_\ell}$	bias
$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}$	pre-activation
$a^{(\ell)} = f(z^{(\ell)})$	activation; f applied elementwise
$\hat{y} = a^{(L)}$	prediction
L	loss on one example, unless a mean is stated
$\delta^{(\ell)} = \nabla_{z^{(\ell)}} L$	backprop signal at layer ℓ
$\odot$	elementwise (Hadamard) product

Home Assignment stores a *batch of rows* and uses $Z = XW + b$. That W is the transpose of the W on this board. The last section of the lecture writes the dictionary.

An affine score

A single unit with input $x \in \mathbb{R}^d$ computes a score and then squashes it:

$$z = w^\top x + b, \qquad a = f(z).$$

- $w \in \mathbb{R}^d$ and $b \in \mathbb{R}$ are the parameters. For $d = 1$, $z = wx + b$.
- z is an affine function of x. The plot against any one coordinate is a straight line.
- f is the only place a curve can enter. If f is the identity, the whole model stays affine.
- Stacking affine maps without a nonlinearity collapses:

$$W_2(W_1 x + b_1) + b_2 = (W_2 W_1)x + (W_2 b_1 + b_2),$$

which is still one affine map.

Logistic regression is a one-neuron network

Binary label $y \in \{0, 1\}$. Choose the sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1), \qquad \hat{y} = \sigma(w^\top x + b).$$

Train with binary cross-entropy

$$L = -\Big(y \log \hat{y} + (1 - y) \log(1 - \hat{y})\Big).$$

Two facts we will prove, then use all semester:

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big),$$
$$\frac{\partial L}{\partial z} = \hat{y} - y.$$

The second identity is why this pair is the default binary head. The factor σ' cancels.

Worked example: logistic forward pass

Take a scalar input. Parameters $w = 2$, $b = -2$, input $x = 1$, label $y = 1$.

$$z = wx + b = 2 \cdot 1 - 2 = 0, \qquad \hat{y} = \sigma(0) = \frac{1}{2}.$$

$$L = -\log \frac{1}{2} = \ln 2 \approx 0.693.$$

Reading the number

$\hat{y} = \frac{1}{2}$ is the same prediction you would make with no information about y. $\ln 2$ is the loss of a coin flip. Random initialization on a balanced binary problem should start near this value.

Worked example: logistic backward pass

Same numbers: $\hat{y} = \frac{1}{2}$, $y = 1$, $x = 1$.

$$\frac{\partial L}{\partial z} = \hat{y} - y = \frac{1}{2} - 1 = -\frac{1}{2}.$$

Chain rule through $z = wx + b$:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot x = -\frac{1}{2} \cdot 1 = -\frac{1}{2}, \qquad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial z} = -\frac{1}{2}.$$

One gradient step with learning rate $\eta > 0$:

$$w \leftarrow w - \eta\Big(-\frac{1}{2}\Big) = 2 + \frac{\eta}{2}.$$

The weight grows, z on this x becomes positive, and $\hat{y}$ moves toward the label 1.

Where $\partial L/\partial z = \hat{y} - y$ comes from

Write $\hat{y} = \sigma(z)$ and differentiate the loss in $\hat{y}$ first.

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})}.$$

Multiply by the sigmoid derivative $\sigma'(z) = \hat{y}(1-\hat{y})$:

$$\frac{\partial L}{\partial z} = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})} \cdot \hat{y}(1-\hat{y}) = \hat{y} - y.$$

Contrast with squared error. If $L = \frac{1}{2}(\hat{y} - y)^2$, the same chain leaves an extra $\sigma'(z)$. In the saturated tails $\sigma' \approx 0$, so the weight update dies even when $\hat{y}$ is confidently wrong.

Three activations and their derivatives

$f(z)$	Formula	$f'(z)$	Where the gradient is tiny
Sigmoid	$\frac{1}{1+e^{-z}}$	$\sigma(z)(1-\sigma(z))$	$\lvert z\rvert$ large; max of σ' is $\frac{1}{4}$ at 0
tanh	$\tanh z$	$1-\tanh^2 z$	$\lvert z\rvert$ large; range is $(-1,1)$
ReLU	$\max(0,z)$	$1[z>0]$	entire half-line $z<0$ (dead unit)

- At $z=0$, ReLU is not classically differentiable. The subgradient is $[0,1]$. In code, use 0.
- Leaky ReLU, $f(z)=\max(\alpha z, z)$ with $\alpha=0.01$, keeps a thin gradient on the negative side.
- Softmax is a map $\mathbb{R}^K \to \mathbb{R}^K$, used on a whole output layer, not on one hidden unit.

Sigmoid derivative, in one line of algebra

$$\sigma(z) = (1 + e^{-z})^{-1}.$$

$$\sigma'(z) = (1 + e^{-z})^{-2} \cdot e^{-z} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z)\big(1 - \sigma(z)\big).$$

Useful values, all exact:

$$\sigma(0) = \frac{1}{2}, \quad \sigma'(0) = \frac{1}{4}, \qquad \sigma'(z) \le \frac{1}{4} \text{ for every } z.$$

So a deep stack of sigmoids multiplies many factors of size at most $1/4$. That product is one reason early deep nets were hard to train. ReLU's derivative on the active side is 1, so it does not shrink the signal by itself.

XOR is not linearly separable

Labels: $(0,0) \mapsto 0$, $(1,0) \mapsto 1$, $(0,1) \mapsto 1$, $(1,1) \mapsto 0$.
Suppose there exist w_1, w_2, b with $w^\top x + b > 0$ on class 1 and < 0 on class 0:

$$\begin{aligned} b &< 0, \\ w_1 + b &> 0, \\ w_2 + b &> 0, \\ w_1 + w_2 + b &< 0. \end{aligned}$$

From the middle two lines, $w_1 > -b$ and $w_2 > -b$. Since $b < 0$, one has $-b > 0$, and

$$w_1 + w_2 + b > (-b) + (-b) + b = -b > 0.$$

That contradicts the last line. No affine score separates XOR. A network with no hidden nonlinearity cannot represent it.

A two-unit ReLU net that realizes XOR

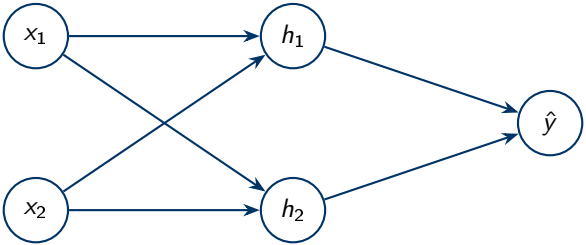
Hidden layer and output, with no extra squashing on y:

$$z^{(1)} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \quad a = \mathrm{ReLU}(z^{(1)}), \quad y = a_1 - 2a_2.$$

x_1	x_2	z_1	z_2	a_1	a_2	y
0	0	0	-1	0	0	0
1	0	1	0	1	0	1
0	1	1	0	1	0	1
1	1	2	1	2	1	0

The second hidden unit turns on only when both inputs are 1, and the output weight -2 cancels the first unit. Depth is doing something a line cannot do.

The network, drawn



- Every hidden unit sees every input. That is a fully connected layer.
- Biases are extra arrows from a constant 1. They are omitted in the picture and kept in the formulas.
- Parameter count of this picture: $W^{(1)}$ has $2 \cdot 2$ entries, $b^{(1)}$ has 2, $W^{(2)}$ has 2, $b^{(2)}$ has 1. Total 9.

Layer recursion

Set $a^{(0)} = x$. For $\ell = 1, \ldots, L-1$,

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \qquad a^{(\ell)} = f^{(\ell)}(z^{(\ell)}).$$

The output layer depends on the task:

- Regression: $f^{(L)}$ is the identity, loss is squared error.
- Binary classification: $f^{(L)} = \sigma$, loss is binary cross-entropy.
- K-class classification: $f^{(L)} = \text{softmax}$, loss is cross-entropy.

Shapes for one example. If layer ℓ has n_ℓ units, $W^{(\ell)}$ is $n_\ell \times n_{\ell-1}$, $b^{(\ell)}$ and $z^{(\ell)}$ and $a^{(\ell)}$ are $n_\ell \times 1$.

Counting parameters

A fully connected map $\mathbb{R}^{n_{\text{in}}} \to \mathbb{R}^{n_{\text{out}}}$ has

$$n_{\text{out}} \cdot n_{\text{in}} + n_{\text{out}}$$

parameters (matrix plus bias).
Example. $784 \to 128 \to 64 \to 10$, as in a small MNIST MLP.

$$\begin{aligned} 784 \cdot 128 + 128 &= 100{,}480, \\ 128 \cdot 64 + 64 &= 8{,}256, \\ 64 \cdot 10 + 10 &= 650. \end{aligned}$$

Total 109,386 parameters. Almost all of them sit in the first matrix, because the input dimension is large. Biases are cheap. This count is a Quiz-style question: write the formula, then the arithmetic.

Three losses

Task	Loss on one example	Gradient you should remember
Regression, $\hat{y} \in \mathbb{R}$	$L = \frac{1}{2}(\hat{y} - y)^2$	$\partial L / \partial \hat{y} = \hat{y} - y$
Binary, $\hat{y} = \sigma(z)$	$L = -\big(y \log \hat{y} + (1 - y) \log(1 - \hat{y})\big)$	$\partial L / \partial z = \hat{y} - y$
K classes, $p = \mathrm{softmax}(z)$, y one-hot	$L = -\sum_{k=1}^{K} y_k \log p_k$	$\nabla_z L = p - y$

A mean over a batch of n examples multiplies every parameter gradient by $1/n$. Keep the same convention in the code and in the derivative, or the learning rate silently changes by a factor n.

Softmax, with fractions

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}.$$

Take logits $z = (\ln 2,\ 0,\ 0)$. Then $e^z = (2, 1, 1)$, the normalizer is 4, and

$$p = \Big(\frac{1}{2},\ \frac{1}{4},\ \frac{1}{4}\Big).$$

If the true class is class 1, then $y = (1, 0, 0)$ and

$$L = -\log \frac{1}{2} = \ln 2, \qquad \nabla_z L = p - y = \Big(-\frac{1}{2},\ \frac{1}{4},\ \frac{1}{4}\Big).$$

Check: the three entries of $p - y$ sum to 0. Softmax probabilities already sum to 1, and a one-hot label sums to 1, so the output-layer error sums to 0. That is a useful debugging test.

Stable binary loss (what you will code)

Forming $\sigma(z)$ and then $\log \hat{y}$ overflows or returns $\log 0$ for large $|z|$. The identity

$$L(z, y) = \max(z, 0) - yz + \log\left(1 + e^{-|z|}\right)$$

is the same binary cross-entropy and stays finite. Home Assignment clips $\hat{y}$ into $[\varepsilon, 1 - \varepsilon]$ as a simpler safeguard; PyTorch's `binary_cross_entropy_with_logits` uses the identity above.

For softmax, subtract the max logit before the exponential:

$$p_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}, \qquad m = \max_j z_j.$$

This does not change p, because e^{-m} cancels, and it keeps the exponentials from overflowing.

Running example: a $2 \to 2 \to 1$ net

One example, both hidden units alive, so every gate in the backward pass is visible.

$$x = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad y = 1, \qquad W^{(1)} = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}, \quad b^{(1)} = \begin{pmatrix} 0 \\ -1/2 \end{pmatrix},$$

$$W^{(2)} = \begin{pmatrix} 1/2 & -1 \end{pmatrix}, \quad b^{(2)} = 0.$$

Hidden activation: ReLU. Output activation: sigmoid. Loss: binary cross-entropy.

We will compute $z^{(1)}, a^{(1)}, z^{(2)}, \hat{y}, L$ exactly, then every parameter gradient, then one gradient step.

Forward pass, layer 1

$$z^{(1)} = W^{(1)}x + b^{(1)} = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ -1/2 \end{pmatrix} = \begin{pmatrix} 1 \\ 1/2 \end{pmatrix}.$$

Both coordinates are positive, so ReLU changes nothing:

$$a^{(1)} = \mathrm{ReLU}(z^{(1)}) = \begin{pmatrix} 1 \\ 1/2 \end{pmatrix}, \qquad \mathrm{ReLU}'(z^{(1)}) = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Keep $z^{(1)}$ in the cache. The backward pass needs the mask $\mathbb{1}[z^{(1)} > 0]$. The value $a^{(1)}$ alone cannot rebuild that mask once a coordinate is zero: both "was negative" and "was exactly zero" produce activation 0.

Forward pass, layer 2 and the loss

$$z^{(2)} = W^{(2)} a^{(1)} + b^{(2)} = \frac{1}{2} \cdot 1 + (-1) \cdot \frac{1}{2} + 0 = 0.$$

$$\hat{y} = \sigma(0) = \frac{1}{2}, \qquad L = -\log \frac{1}{2} = \ln 2.$$

Cache for backprop

Store $(x,\ z^{(1)},\ a^{(1)},\ z^{(2)})$. Home Assignment returns the same four arrays, batched. Anything you can recompute from these is optional; the pre-activations are not optional for ReLU.

Scalar computation graph

Warm-up, before any network. Let $w = 2$, $x = 3$, $y = 4$, and

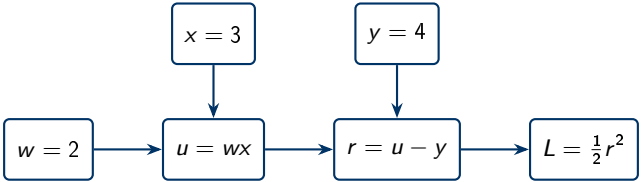
$$u = wx, \qquad r = u - y, \qquad L = \tfrac{1}{2} r^2.$$

Forward: $u = 6$, $r = 2$, $L = 2$.
Backward, local derivative at each node, then multiply along the path:

$$\begin{aligned}
\frac{\partial L}{\partial r} &= r = 2, \\
\frac{\partial L}{\partial u} &= \frac{\partial L}{\partial r} \cdot 1 = 2, \\
\frac{\partial L}{\partial w} &= \frac{\partial L}{\partial u} \cdot x = 2 \cdot 3 = 6, \\
\frac{\partial L}{\partial x} &= \frac{\partial L}{\partial u} \cdot w = 2 \cdot 2 = 4.
\end{aligned}$$

Direct check: $L = \frac{1}{2}(wx - y)^2$ gives $\partial L / \partial w = (wx - y)x = 6$.

The same graph, as a picture



Each edge carries a **local** derivative ($\partial u/\partial w = x$, $\partial r/\partial u = 1$, $\partial L/\partial r = r$). The **global** derivative is the product of local derivatives along the unique path. When a node has two parents that both depend on one variable, you add the two path products. That sum is the multivariate chain rule.

Outer products are the chain rule in index form

One layer, $z = Wx + b$, with $z \in \mathbb{R}^m$, $x \in \mathbb{R}^d$, $W \in \mathbb{R}^{m \times d}$. Suppose L depends on z and you already know $\delta = \nabla_z L$.

$$z_i = \sum_{j=1}^{d} W_{ij} x_j + b_i.$$

Differentiate one entry:

$$\frac{\partial L}{\partial W_{ij}} = \frac{\partial L}{\partial z_i} \cdot x_j = \delta_i \, x_j.$$

Those entries are exactly the outer product

$$\nabla_W L = \delta \, x^\top, \qquad \nabla_b L = \delta, \qquad \nabla_x L = W^\top \delta.$$

Memorize these three. Every fully connected backward step in the course is this block, plus an elementwise f'.

The δ recursion

At the output, for binary cross-entropy with a sigmoid,

$$\delta^{(L)} = \hat{y} - y.$$

For a hidden layer with elementwise activation f,

$$\delta^{(\ell)} = \left(\left(W^{(\ell+1)} \right)^{\top} \delta^{(\ell+1)} \right) \odot f'(z^{(\ell)}).$$

Then the parameter gradients at every layer, including the output, are

$$\nabla_{W^{(\ell)}} L = \delta^{(\ell)} \left(a^{(\ell-1)} \right)^{\top}, \qquad \nabla_{b^{(\ell)}} L = \delta^{(\ell)}.$$

Why the transpose. $W^{(\ell+1)}$ is shaped (next $\times$ current). Multiplying by its transpose sends a vector of "next" length back to "current" length. The Hadamard product with f' applies the local derivative of the activation at this layer.

Backward pass on the running example

We have $\hat{y} = \frac{1}{2}$, $y = 1$, $a^{(1)} = (1,\ 1/2)^\top$, and $\mathrm{ReLU}'(z^{(1)}) = (1, 1)^\top$.

$$\delta^{(2)} = \hat{y} - y = -\frac{1}{2}.$$

$$\nabla_{W^{(2)}} L = \delta^{(2)} \left(a^{(1)}\right)^\top = -\frac{1}{2} \begin{pmatrix} 1 & 1/2 \end{pmatrix} = \begin{pmatrix} -1/2 & -1/4 \end{pmatrix}, \qquad \nabla_{b^{(2)}} L = -\frac{1}{2}.$$

$$\delta^{(1)} = \left(W^{(2)}\right)^\top \delta^{(2)} = \begin{pmatrix} 1/2 \\ -1 \end{pmatrix} \left(-\frac{1}{2} \right) = \begin{pmatrix} -1/4 \\ 1/2 \end{pmatrix}.$$

The ReLU mask is all ones, so it leaves $\delta^{(1)}$ unchanged.

Parameter gradients at layer 1

$$\nabla_{W^{(1)}} L = \delta^{(1)} x^\top = \begin{pmatrix} -1/4 \\ 1/2 \end{pmatrix} \begin{pmatrix} 1 & 0 \end{pmatrix} = \begin{pmatrix} -1/4 & 0 \\ 1/2 & 0 \end{pmatrix}, \qquad \nabla_{b^{(1)}} L = \begin{pmatrix} -1/4 \\ 1/2 \end{pmatrix}.$$

Read the zeros

$x_2 = 0$, so the second column of $\nabla_{W^{(1)}} L$ is zero. Those two weights did not affect $z^{(1)}$ on this particular input. A zero gradient here is correct; it is not a dead ReLU. Dead ReLU zeros a *row* of the gradient, through f', even when the input coordinate is nonzero. Problem 3 is that case.

One gradient step, then the loss falls

Use $\eta = 1/10$. Only the nonzero gradients move.

$$W^{(2)} \leftarrow \begin{pmatrix} 11/20 & -39/40 \end{pmatrix}, \qquad b^{(2)} \leftarrow 1/20,$$

$$W^{(1)} \leftarrow \begin{pmatrix} 41/40 & -1 \\ 19/20 & 1 \end{pmatrix}, \qquad b^{(1)} \leftarrow \begin{pmatrix} 1/40 \\ -11/20 \end{pmatrix}.$$

Forward again on the same x:

$$z^{(1)} = \begin{pmatrix} 21/20 \\ 2/5 \end{pmatrix}, \quad a^{(1)} = z^{(1)}, \quad z^{(2)} = \frac{19}{80} = 0.2375.$$

$$\hat{y} = \sigma(19/80) \approx 0.559, \qquad L \approx 0.581 \quad (\text{was } \ln 2 \approx 0.693).$$

Full arithmetic is in the lecture notes. The direction of the step is what matters here: $\hat{y}$ moved from 0.50 toward the label 1.

From one column to a batch of rows

Home Assignment uses batch size n, examples as rows, and a *mean* loss.

	Board (one column)	NumPy (batch of rows)
Forward	$z = Wx + b$	$Z = XW_{\text{code}} + b$
Weights	W is (out $\times$ in)	$W_{\text{code}} = W^\top$ is (in $\times$ out)
Output error	$\delta = \hat{y} - y$	$dZ_2 = (\hat{Y} - Y)/n$
∇_W	$\delta\, a^\top$	$dW_{\text{code}} = A^\top dZ$
To the layer below	$W^\top \delta$	$dA = dZ\, W_{\text{code}}^\top$
ReLU	$\delta \odot 1[z > 0]$	$dZ_1 = dA \odot 1[Z_1 > 0]$

On the running example, $n = 1$, the board gradient $\nabla_{W^{(2)}} L = (-1/2,\ -1/4)$ and the code gradient is that row, transposed, as a column. Same linear map.

The training step you will implement

```
for step in range(T):
    yhat, cache = forward(X, params)      # cache holds Z1, not only A1
    grads = backward(cache, yhat, y)      # hand-written, no autograd
    params = [p - eta * g for p, g in zip(params, grads)]
```

- Check shapes before checking values. A bias of shape $(h,)$ versus $(1, h)$ is the usual first bug.
- Check gradients against PyTorch on *the same weights*, with a relative error below 10^{-5}, before you train.
- A flat loss is often a dead hidden layer, a learning rate at 0, or a forgotten $1/n$, not a mysterious optimizer. Optimizers are Week 2.

In-class problems

Work on paper. Exact fractions are enough; a decimal is only needed if you choose to approximate σ.

1. Sigmoid values and the size of σ'. 4 minutes
2. Parameter count of a $4 \to 3 \to 2$ network. 3 minutes
3. Forward and backward through a dead ReLU unit. 12 minutes
4. Softmax cross-entropy gradient, and a sum-to-zero check. 6 minutes

Each problem slide has a button to its solution. The written sheet (`CSE752_W01_Lecture_Notes.tex`) has eight problems, full derivations, and the one-step recomputation.

Problem 1. Sigmoid

1. Compute $\sigma(0)$ and $\sigma'(0)$ exactly.
2. Show that $\sigma'(z) \leq 1/4$ for every real z, with equality only at $z = 0$.
3. A hidden layer uses a sigmoid, and its pre-activation on a training point is $z = 8$. Give a bound on that unit's factor in the backward pass, and say what this does to learning.

▸ Solution 1

Problem 2. Shapes and parameter count

A network is $4 \to 3 \to 2$. Hidden activation ReLU, output left as logits.

1. Write the shape of $W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}$ in the **board** convention (out $\times$ in).
2. How many scalar parameters are there?
3. A batch of 32 rows uses the **code** convention. Write the shape of X, of $W^{(1)}_{\text{code}}$, and of $Z^{(1)}$.

▸ Solution 2

Problem 3. A dead unit

$$x = \begin{pmatrix} 1 \\ 1 \end{pmatrix},\ y = 0, \quad W^{(1)} = I_2,\ b^{(1)} = \begin{pmatrix} -1/2 \\ -3/2 \end{pmatrix},\ W^{(2)} = \begin{pmatrix} 2 & 2 \end{pmatrix},\ b^{(2)} = -1.$$

Hidden activation ReLU, output sigmoid, loss binary cross-entropy.

1. Compute $z^{(1)}, a^{(1)}, z^{(2)}, \hat{y}$, and L.
2. Compute $\delta^{(2)}$, $\nabla_{W^{(2)}} L$, $\nabla_{b^{(2)}} L$, $\delta^{(1)}$, $\nabla_{W^{(1)}} L$, and $\nabla_{b^{(1)}} L$.
3. Which hidden unit is dead, and which entries of the gradients are zero because of that?

▸ Solution 3

Problem 4. Softmax

Logits $z = (0,\ \ln 3)$, one-hot label $y = (0,\ 1)$.

1. Compute $p = \text{softmax}(z)$ and the cross-entropy L exactly.
2. Compute $\nabla_z L = p - y$.
3. Check that the entries of p sum to 1 and the entries of $\nabla_z L$ sum to 0.
4. For $K = 2$, softmax plus cross-entropy is the same model as a sigmoid on the logit difference. Identify that difference here, and check that $\partial L / \partial z_2$ equals $\sigma(z_2 - z_1) - 1$.

▸ Solution 4

What this week asks of you

- **Read,** in this order if time is short: Prince Ch. 3–4 for the mechanics, Goodfellow Ch. 6 for the vector backprop. Nielsen Ch. 2 is an optional scalar walkthrough.
- **Home Assignment W1.** Build $2 \to 16 \to 1$ in NumPy on two-moons, implement the backward pass from today's formulas, match PyTorch to relative error $< 10^{-5}$, train, draw the decision boundary.
- **Quiz 1** can ask for a derivative, a shape, a short chain-rule step, or a verbal reason (why the ReLU cache stores z, why BCE with a sigmoid simplifies). The four problems are the right practice.

Equations worth leaving with

$$
\begin{aligned}
z^{(\ell)} &= W^{(\ell)} a^{(\ell-1)} + b^{(\ell)} \\
\sigma'(z) &= \sigma(z)\big(1 - \sigma(z)\big) \\
\frac{\partial L_{\mathrm{BCE}}}{\partial z} &= \sigma(z) - y \\
\nabla_z L_{\mathrm{CE}} &= \mathsf{softmax}(z) - y \\
\delta^{(\ell)} &= \big((W^{(\ell+1)})^\top \delta^{(\ell+1)}\big) \odot f'(z^{(\ell)}) \\
\nabla_{W^{(\ell)}} L &= \delta^{(\ell)} (a^{(\ell-1)})^\top
\end{aligned}
$$

References

- Goodfellow, Bengio, Courville. *Deep Learning*, Ch. 6. `https://www.deeplearningbook.org/`
- Prince. *Understanding Deep Learning*, Ch. 3–4. `https://udlbook.github.io/udlbook/`
- Zhang, Lipton, Li, Smola. *Dive into Deep Learning*, MLP chapter. `https://d2l.ai/`
- Nielsen. *Neural Networks and Deep Learning*, Ch. 2 (scalar backprop). `http://neuralnetworksanddeeplearning.com/`
- Cybenko (1989); Hornik (1991): a wide one-hidden-layer net can approximate continuous functions on a compact set. This is an existence result. It does not choose the width or make the optimization easy.

Solution 1

1. $\sigma(0) = 1/2$ and $\sigma'(0) = (1/2)(1/2) = 1/4$.
2. Let $s = \sigma(z) \in (0,1)$. Then $\sigma'(z) = s(1-s)$. The quadratic $s(1-s)$ on $(0,1)$ has maximum $1/4$ at $s = 1/2$, i.e. only at $z = 0$.
3. $\sigma(8) = 1/(1+e^{-8})$ is extremely close to 1, so

$$\sigma'(8) = \sigma(8)\big(1-\sigma(8)\big) < e^{-8} \approx 3.4 \times 10^{-4}.$$

The backward factor through that unit is smaller than 10^{-3}. A gradient step on the incoming weights of this unit is negligible: the unit is saturated.

Back to Problem 1

Solution 2

1. Board convention: $W^{(1)} \in \mathbb{R}^{3\times 4}$, $b^{(1)} \in \mathbb{R}^3$, $W^{(2)} \in \mathbb{R}^{2\times 3}$, $b^{(2)} \in \mathbb{R}^2$.
2. Count: $(3 \cdot 4 + 3) + (2 \cdot 3 + 2) = 15 + 8 = 23$.
3. Code convention, batch of 32: $X \in \mathbb{R}^{32\times 4}$, $W^{(1)}_{\text{code}} \in \mathbb{R}^{4\times 3}$, $Z^{(1)} \in \mathbb{R}^{32\times 3}$.

Back to Problem 2

Solution 3 (forward)

$$z^{(1)} = \begin{pmatrix}1\\1\end{pmatrix} + \begin{pmatrix}-1/2\\-3/2\end{pmatrix} = \begin{pmatrix}1/2\\-1\end{pmatrix}, \qquad a^{(1)} = \mathrm{ReLU}(z^{(1)}) = \begin{pmatrix}1/2\\0\end{pmatrix}.$$

$$z^{(2)} = 2 \cdot \frac{1}{2} + 2 \cdot 0 - 1 = 0, \qquad \hat{y} = \sigma(0) = \frac{1}{2}.$$

Label $y = 0$, so

$$L = -(1-y)\log(1-\hat{y}) = -\log\frac{1}{2} = \ln 2.$$

Unit 2 is dead: $z_2^{(1)} = -1 < 0$. ▸ Backward pass ▸ Back to Problem 3

Solution 3 (backward)

$$\delta^{(2)} = \hat{y} - y = \frac{1}{2}, \quad \nabla_{W^{(2)}} L = \frac{1}{2}\begin{pmatrix} 1/2 & 0 \end{pmatrix} = \begin{pmatrix} 1/4 & 0 \end{pmatrix}, \quad \nabla_{b^{(2)}} L = \frac{1}{2}.$$

$$\left(W^{(2)}\right)^{\top} \delta^{(2)} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad \mathrm{ReLU}'(z^{(1)}) = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad \delta^{(1)} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

$$\nabla_{W^{(1)}} L = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}, \qquad \nabla_{b^{(1)}} L = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

The dead unit freezes its incoming row, its bias, and its outgoing weight (the outgoing weight also sees $a_2 = 0$). Unit 1 still learns. Back to Problem 3

Solution 4

$e^z = (1,\ 3)$, normalizer 4, so $p = (1/4,\ 3/4)$ and $L = -\log(3/4) = \log(4/3)$.

$$\nabla_z L = p - y = \left(\frac{1}{4},\ \frac{3}{4} - 1\right) = \left(\frac{1}{4},\ -\frac{1}{4}\right).$$

Sum of p is 1. Sum of $\nabla_z L$ is 0.
Logit difference: $z_2 - z_1 = \ln 3$.

$$\sigma(\ln 3) = \frac{1}{1 + e^{-\ln 3}} = \frac{1}{1 + 1/3} = \frac{3}{4}, \qquad \sigma(\ln 3) - 1 = \frac{3}{4} - 1 = -\frac{1}{4},$$

which equals $\partial L / \partial z_2$. Class 2's probability under the sigmoid of the difference matches p_2.

Back to Problem 4